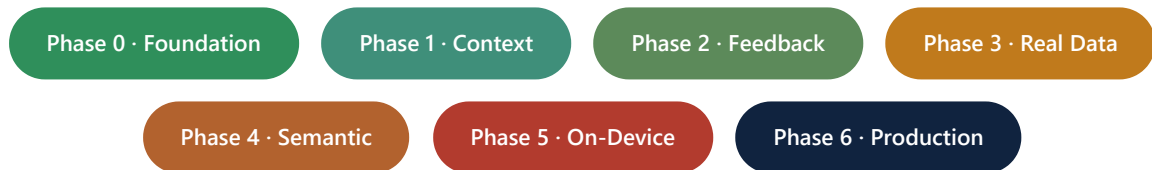


From Prototype to Production — and Beyond

A sequential plan, with requirements, from the smallest next change
to a fully production-ready system



1. How to read this roadmap

This document lays out seven sequential phases — each one buildable on top of the last, starting from the model retrain already committed to this repository and ending at a genuinely production-ready on-device product. Every phase lists: what changes, why it's next, and exactly what you'd need (data, engineering time, infrastructure, skills) to do it. A final note covers what this project would look like with no resource constraints at all.

Each phase carries three badges:

- **LOW EFFORT** / **MEDIUM EFFORT** / **HIGH EFFORT** — engineering effort relative to the others here, not absolute difficulty.
- **TIMEFRAME** — a rough calendar estimate for one focused engineer (or small team where noted).
- **MILESTONE** — marks the phase that represents "production-ready."

2. Roadmap at a glance



Each phase is additive — nothing in a later phase requires undoing an earlier one.
Phases 0-2 need no new infrastructure; Phases 3-6 progressively add people, data and hardware.

Figure 1 — the seven-phase roadmap, effort and rough timeframe per phase

3. Phase 0 — Foundation Hardening

Make the current classifier's fixes solid before building on them

LOW EFFORT

2-3 WEEKS

This phase closes out loose ends from the classifier enhancement already committed: the disclosed "vague future risk" trade-off, and the lack of any automated guard against future regressions.

- Properly resolve the disclosed regression on referent-free risk phrasing ("that might be a problem later") — likely needs a handful of real or carefully-designed synthetic examples distinct enough from the "task"/"decision" augmentation to not cross-contaminate them again.
- Expand the hand-written scenario suite beyond 22 turns — more multi-speaker conversations, more edge cases per category, so future retrains have a bigger safety net.
- Add a **regression gate**: a script that fails if a retrain drops the scenario-suite score or raises the true-drop rate above the last committed baseline — turns the manual verification done for this fix into something automatic.

Data

- A few dozen new synthetic phrases
- 5-10 new hand-written scenarios

Engineering

- Notebook/data science time only
- A small Python regression-check script

Infra

- None new — reuses `backend/simulator.py`

Skills

- Same skillset as the current codebase (pandas, sklearn)

4. Phase 1 — Context-Aware Classification

Let the model use the previous turn — done right this time

MEDIUM EFFORT

3-4 WEEKS

A naive version of this (concatenating previous + current text into one input) was tried during the original build and reverted — it contaminated short ordinary lines with the previous line's vocabulary. The correct version keeps the current-utterance signal clean and adds context as a *separate* feature, not a blended one.

Tried & reverted



Phase 1 approach

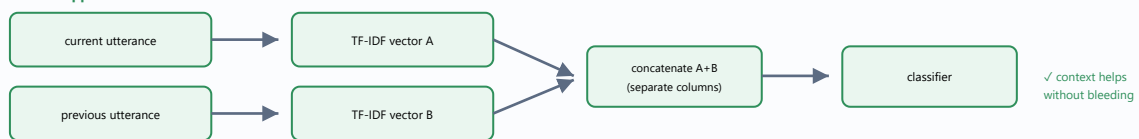


Figure 2 — why the previous attempt failed, and the separate-feature-columns approach for Phase 1

Data

- Multi-turn synthetic conversations (not just isolated utterances)
- Scenario suite turns that specifically depend on the prior line

Engineering

- Feature-union changes in `notebooks/02`
- `backend/gatekeeper.py` already tracks a context ring buffer — reuse it as the feature source

Infra

- None new

Skills

- Feature engineering, careful ablation testing (to avoid repeating the original contamination bug)

5. Phase 2 — Feedback-Aware Session State

Let downstream tell the Gatekeeper when a moment has been handled

MEDIUM EFFORT

2-3 WEEKS

Today, duplicate suppression only knows about moments the Gatekeeper itself forwarded — it has no idea if downstream ever actually resolved them. This phase adds a lightweight, fully local feedback channel so that a resolved moment ("stove's off now, thanks") doesn't get re-forwarded on every restatement, and starts collecting the implicit signal (was a forward acted on or dismissed?) that Phase 6's adaptive thresholds will eventually need.

- New endpoint, e.g. `POST /api/feedback`, marking a previously-forwarded moment as handled/dismissed.
- Extend `GatekeeperEngine`'s duplicate memory with a "resolved" flag per entry.
- Start logging (locally, not centrally) forward-vs-dismissed counts per session as raw material for Phase 6.

Data

- None new — this is a logic/schema change

Engineering

- New Pydantic schema + FastAPI route
- `GatekeeperEngine` state extension

Infra

- None new (still local, no server-side storage)

Skills

- Same backend stack already in use (FastAPI, Pydantic)

6. Phase 3 — Real-Data Pilot & Active Learning Loop

Graduate from synthetic data to real, consented conversation data

MEDIUM-HIGH EFFORT

6-8 WEEKS, THEN ONGOING

This is the first phase that needs people outside engineering: consented pilot users and someone doing annotation. The existing synthetic-data model is used to bootstrap the pool of candidates worth labelling.

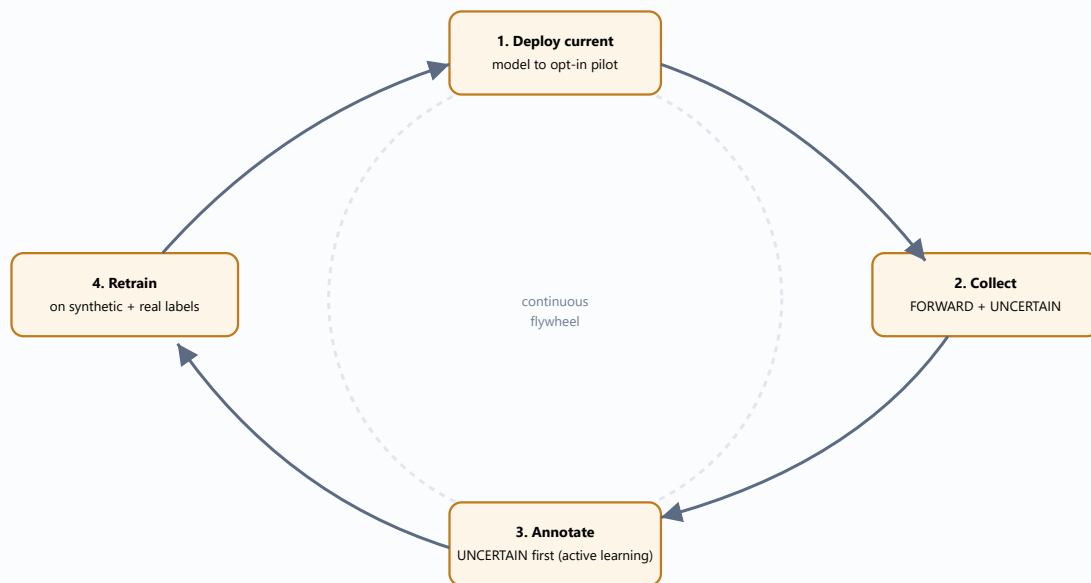


Figure 3 — the active-learning flywheel: label the model's own uncertainty band first, for the cheapest accuracy gain per label

- Opt-in pilot cohort with clear, revocable consent.
- Lightweight internal annotation tool: one fragment + 3 lines of prior context, two questions ("should this reach the assistant?" and "which moment type?") — matches the process already designed in `docs/data_strategy.md` §5.
- PII scrubbing before any storage; fragments only, never full-conversation export.
- Track inter-annotator agreement — low-agreement items become `UNCERTAIN` training targets rather than being forced into a class.

Data

- Consented pilot users (even 10-20 is a meaningful start)
- PII scrubbing pipeline

Engineering

- Small internal annotation web app
- Active-learning item-selection script

Infra

- Minimal secure storage for consented fragments only

People

- 1-2 part-time annotators

- Privacy/legal sign-off on the consent flow

7. Phase 4 — Semantic Second-Stage Re-scorer

Add paraphrase understanding, without leaving the 25 MB budget

HIGH EFFORT

4-6 WEEKS

The cheap linear model handles the confident majority of fragments in ~1-2 ms. This phase adds a small quantised encoder that *only* runs on the minority landing in the **UNCERTAIN** band — the place a semantic model earns its keep, without paying its cost on every single fragment.

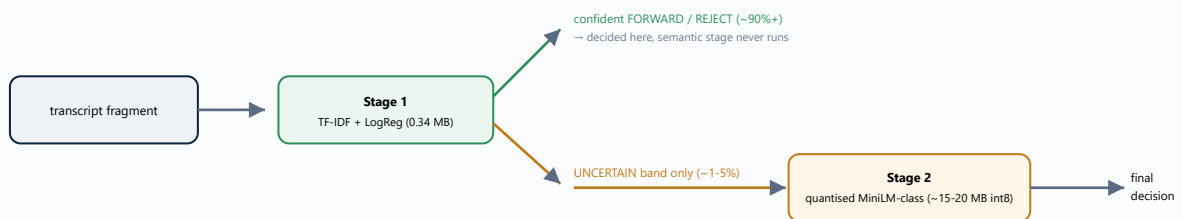


Figure 4 — two-stage cascade: the semantic model only runs on the grey band, keeping total assets under 25 MB

- Decide the model-provenance question up front: train a small encoder from scratch (keeps the original "no external pretrained models" constraint) vs. adopt a permissively-licensed small pretrained encoder (faster, but changes the submission's compliance story).
- Quantise to int8, export via ONNX, and budget-check: current assets (0.34 MB) + encoder (~15-20 MB) must stay under 25 MB.
- Re-run the full evaluation suite with the cascade active — the win to prove is fewer paraphrase-blind misses in the **UNCERTAIN** band specifically, not a change to the Stage-1 numbers.

Data

- Reuses existing labelled data; may need paraphrase-focused eval examples

Engineering

- ONNX export/quantisation tooling
- Cascade logic in `backend/gatekeeper.py`

Infra

- ONNX Runtime (CPU) for serving; a GPU is only needed to train/distill, not to run

Skills

- Model compression/quantisation experience

8. Phase 5 — On-Device Android/Kotlin Port

Move off the FastAPI prototype and onto the actual target device

HIGH EFFORT

6-8 WEEKS

`docs/architecture.md` §5 already scopes this: the sparse linear model is two vocabulary hash maps, a coefficient matrix, and a softmax — reimplementable directly in Kotlin in roughly 200 lines, with no ML runtime dependency at all. The regex back-channel filter, threshold gate, duplicate cosine check and ring buffers port over just as directly.

- Export vectorizer + classifier weights to a flat binary/JSON (<1 MB).
- Reimplement transform → dot-product → softmax in Kotlin; port the deterministic layers unchanged.
- If Phase 4 shipped, embed ONNX Runtime Mobile (~5 MB APK overhead, outside the 25 MB model budget) for the semantic stage only.
- Battery/latency benchmarking on a real mid-range device, not just a laptop CPU proxy.

Data

- None new — this is a faithful port, not a retrain

Engineering

- Android/Kotlin engineer
- Cross-checking harness comparing Kotlin output vs. the Python engine bit-for-bit on the scenario suite

Infra

- A small device lab spanning a few chipsets/RAM tiers
- Battery profiling tooling (Android Studio Profiler / Battery Historian)

Skills

- Kotlin, Android platform APIs, on-device performance profiling

9. Phase 6 — Production Hardening & Scale-Out (the production-ready milestone)

Everything a real, shipping product needs beyond a working model

HIGH EFFORT

2-3 MONTHS

MILESTONE

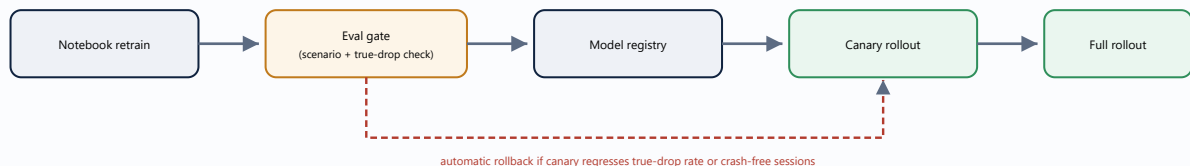


Figure 5 — the retrain-to-rollout pipeline that turns Phase 0's regression gate into a real release process

- **Multi-language expansion** — Hinglish first (the char n-gram design was chosen partly for this), then further languages, each needing its own template/vocabulary pass and evaluation.
- **Per-user adaptive thresholds** — graduate Phase 2's raw feedback logs into an actual learned personalization layer (implicit feedback nudges the forward threshold per user).
- **Observability** — aggregated, privacy-safe telemetry on decision latency, crash rates, and forward/reject/uncertain ratios in the field (never raw transcript content).
- **CI/CD** — the pipeline in Figure 5, wired to real infrastructure, with staged canary rollout and automatic rollback.
- **Privacy & compliance** — formal Data Protection Impact Assessment given this is an always-on ambient-audio-adjacent product; on-device encryption of any cached fragments; a clear data retention policy.
- **A/B testing infrastructure** for threshold and model changes before full rollout.

Data

- Aggregated field telemetry (opt-in, privacy-preserving)

Engineering

- SRE / mobile-infra engineer
- CI infra (e.g. GitHub Actions) wired to the notebook + simulator gate

Infra

- Model registry, canary rollout tooling, telemetry backend

People

- Privacy/legal counsel for the DPIA
- On-call rotation once this is a real release

10. Production readiness checklist

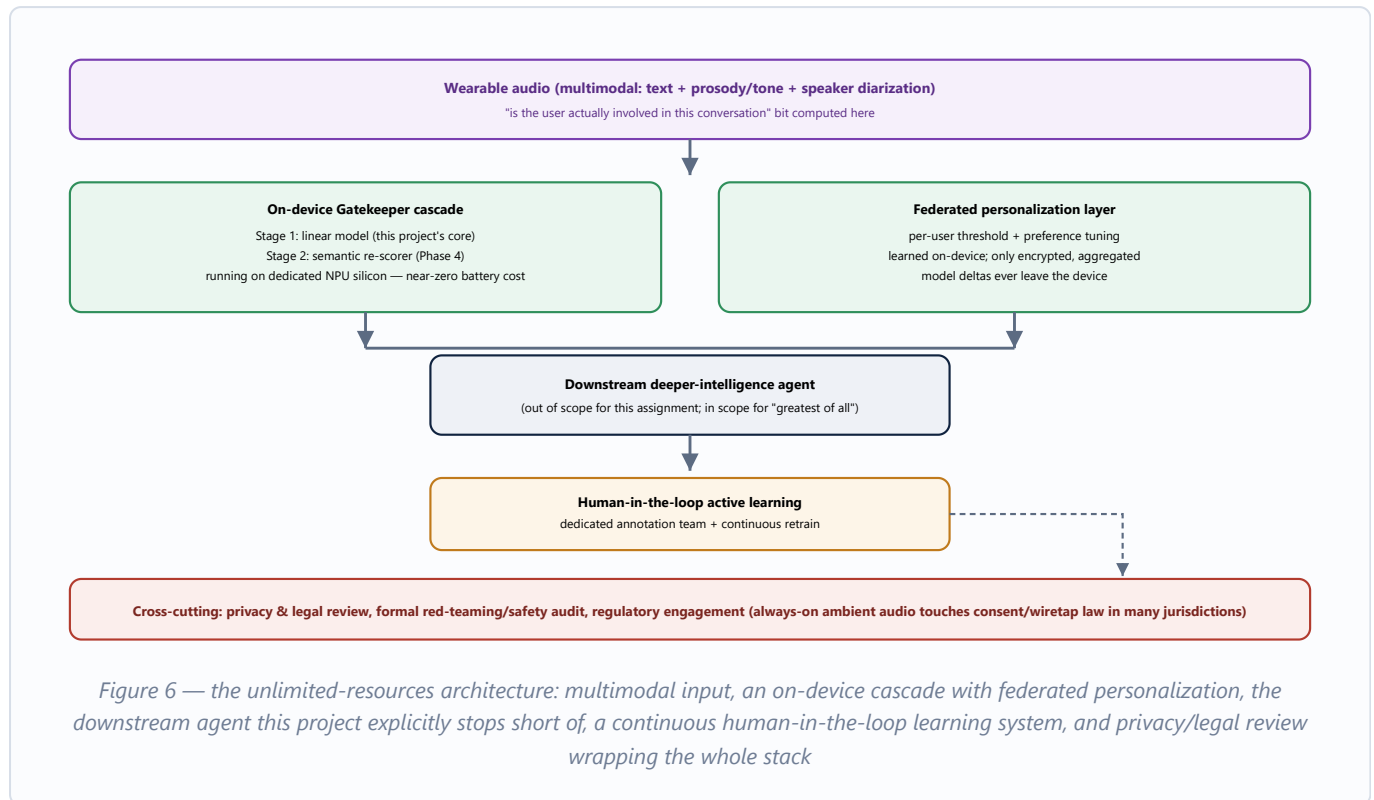
Area	Done by which phase	Status if all phases complete
Core model quality (binary gate)	Phase 0 (already underway)	Automated regression gate in place
Multi-turn context	Phase 1	Context used safely, no contamination
Feedback loop	Phase 2 → 6	From logging-only to full personalization
Real-world data	Phase 3	Continuous active-learning pipeline
Paraphrase robustness	Phase 4	Cascade re-scorer live, still <25 MB
Runs on target hardware	Phase 5	Native Kotlin, benchmarked on real devices
Release safety net	Phase 6	CI gate + canary + rollback
Privacy/compliance sign-off	Phase 6	DPIA complete, retention policy live
Multi-language support	Phase 6	Hinglish shipped, others planned

11. Note — the greatest possible version of this project, with unlimited resources

IF BUDGET, HEADCOUNT AND TIME WERE NOT CONSTRAINTS

Every phase above is scoped to be achievable by a small team on a realistic timeline. With no resource constraints at all, the project's ceiling is much higher than "a good Gatekeeper" — it becomes the foundation of a genuinely trustworthy, always-on ambient intelligence layer. Here is the structure that would get there, and what it would actually take.

The ideal structure



What it would take

Team

- ML researchers (multimodal + on-device efficiency)
- Mobile/embedded engineers (Android, NPU/silicon integration)
- Privacy & legal counsel (dedicated, not advisory)
- UX researchers (trust, consent, notification design)
- A standing annotation team
- SRE/on-call for a real always-on service

Data & learning infrastructure

- Federated learning infrastructure (e.g. secure aggregation, differential privacy) so personalization never centralizes raw audio/text
- A continuously-running active-learning loop, not a one-time pilot
- Multilingual data collection across target markets

Hardware

- NPU/custom-silicon partnership for always-on inference at near-zero battery cost
- A large, diverse on-device benchmarking lab

Trust & safety

- Formal adversarial red-teaming — this listens to everything around the user, so the stakes for getting it wrong are unusually high
- Regulatory engagement per market (consent/wiretapping law varies widely for always-on audio)
- A published, auditable privacy policy and data retention model

The throughline across every phase in this document, resourced or not, is the same one this project already follows: measure everything, disclose trade-offs honestly, and never let a confident-sounding model make a decision it hasn't earned the right to make.

